```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 43s 49ms/step - accuracy: 0.9481 - loss: 0.1765 - val_accuracy: 0.9770 - val_loss: 0.0736
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9834 - loss: 0.0527 - val_accuracy: 0.9852 - val_loss: 0.0473
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9888 - loss: 0.0361 - val_accuracy: 0.9893 - val_loss: 0.0381
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9919 - loss: 0.0261 - val_accuracy: 0.9885 - val_loss: 0.0379
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9935 - loss: 0.0193 - val_accuracy: 0.9895 - val_loss: 0.0354
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - accuracy: 0.9946 - loss: 0.0162 - val_accuracy: 0.9895 - val_loss: 0.0352
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - accuracy: 0.9960 - loss: 0.0123 - val_accuracy: 0.9893 - val_loss: 0.0394
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - accuracy: 0.9972 - loss: 0.0091 - val_accuracy: 0.9918 - val_loss: 0.0344
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9970 - loss: 0.0083 - val_accuracy: 0.9895 - val_loss: 0.0524
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 47ms/step - accuracy: 0.9980 - loss: 0.0073 - val_accuracy: 0.9908 - val_loss: 0.0492
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9909 - loss: 0.0329
Test accuracy: 0.9908999800682068
```

```
import matplotlib.pyplot as plt

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_767/270378434.py in <cell line: 0>()
      3 # Accuracy Graph
      4 plt.figure()
----> 5 plt.plot(history.history['accuracy'])
      6 plt.plot(history.history['val_accuracy'])
      7 plt.title('Model Accuracy')

NameError: name 'history' is not defined

<Figure size 640x480 with 0 Axes>
```

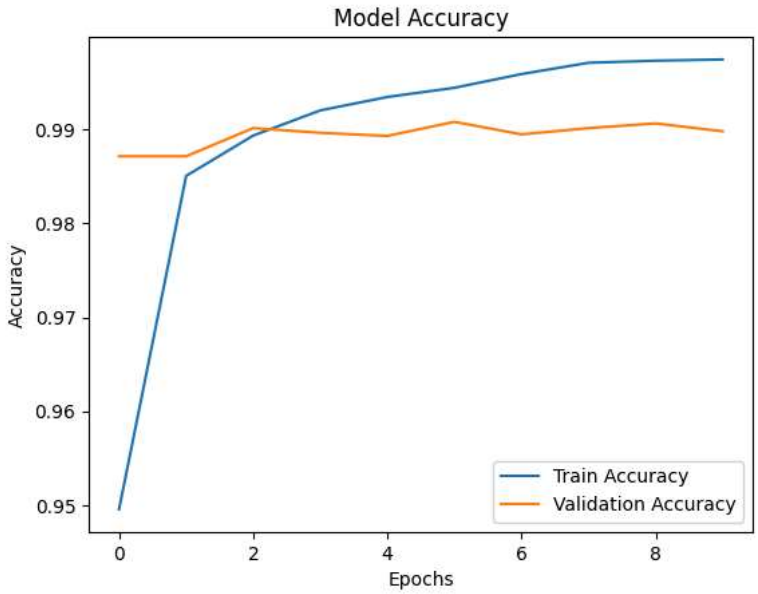
```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

history = model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

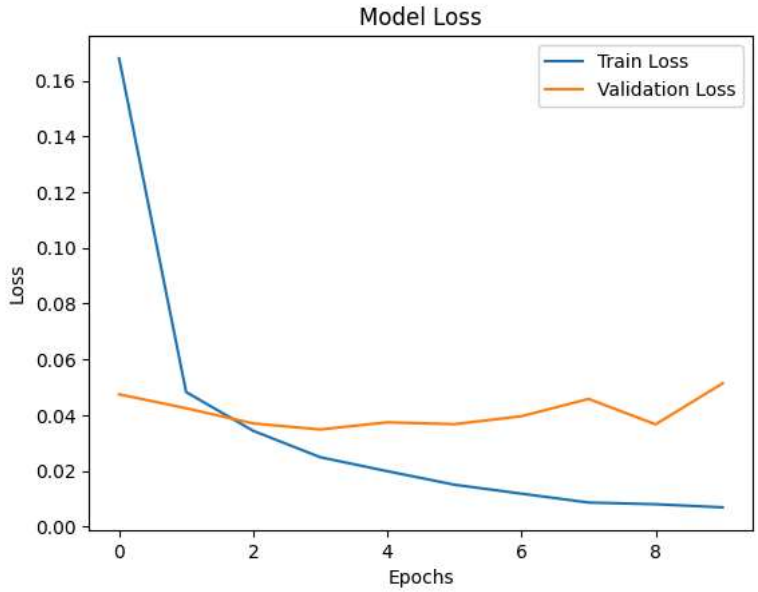
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dense_3 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 43s 49ms/step - accuracy: 0.9496 - loss: 0.1680 - val_accuracy: 0.9872 - val_loss: 0.0475
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9851 - loss: 0.0483 - val_accuracy: 0.9872 - val_loss: 0.0424
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9894 - loss: 0.0343 - val_accuracy: 0.9902 - val_loss: 0.0370
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9921 - loss: 0.0249 - val_accuracy: 0.9897 - val_loss: 0.0348
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9935 - loss: 0.0199 - val_accuracy: 0.9893 - val_loss: 0.0374
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 48ms/step - accuracy: 0.9945 - loss: 0.0150 - val_accuracy: 0.9908 - val_loss: 0.0367
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9959 - loss: 0.0118 - val_accuracy: 0.9895 - val_loss: 0.0396
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9971 - loss: 0.0086 - val_accuracy: 0.9902 - val_loss: 0.0458
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9974 - loss: 0.0080 - val_accuracy: 0.9907 - val_loss: 0.0367
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9975 - loss: 0.0069 - val_accuracy: 0.9898 - val_loss: 0.0514
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9886 - loss: 0.0446
Test accuracy: 0.9886000156402588
```

```
import matplotlib.pyplot as plt

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```





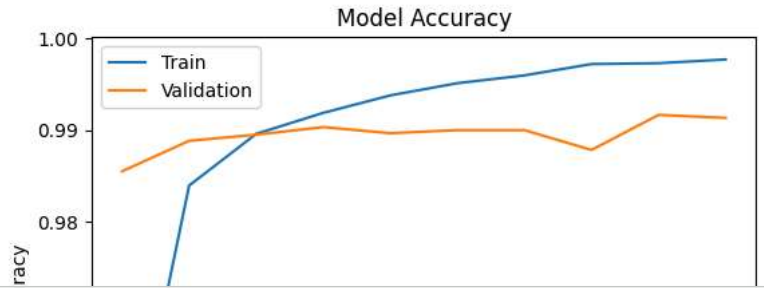

```
# 1. Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import matplotlib.pyplot as plt
import numpy as np

# 2. Load Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 3. Normalization
X_train = X_train / 255.0
X_test = X_test / 255.0

# 4. Train/Test already split in MNIST

# 5. Reshape for CNN (important)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# 6. Build Model (Conv → Pool → Flatten → Dense)
model = Sequential()

# Convolution
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
```

```
# Pooling
model.add(MaxPooling2D((2,2)))

# Convolution
model.add(Conv2D(64, (3,3), activation='relu'))

# Pooling
model.add(MaxPooling2D((2,2)))

# Flatten
model.add(Flatten())

# Dense
model.add(Dense(128, activation='relu'))

# Output Dense
model.add(Dense(10, activation='softmax'))

# 7. Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 8. Summary
model.summary()

# 9. Training
history = model.fit(X_train, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)

# 10. Evaluation
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 11. Graphs (Loss & Accuracy)

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dense_4 (Dense)	(None, 128)	204,928
dense_5 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 43s 49ms/step - accuracy: 0.9486 - loss: 0.1677 - val_accuracy: 0.9855 - val_loss: 0.0521
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9839 - loss: 0.0512 - val_accuracy: 0.9888 - val_loss: 0.0393
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9896 - loss: 0.0339 - val_accuracy: 0.9895 - val_loss: 0.0360
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9919 - loss: 0.0252 - val_accuracy: 0.9903 - val_loss: 0.0368
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9938 - loss: 0.0190 - val_accuracy: 0.9897 - val_loss: 0.0345
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9951 - loss: 0.0149 - val_accuracy: 0.9900 - val_loss: 0.0387
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9960 - loss: 0.0121 - val_accuracy: 0.9900 - val_loss: 0.0379
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9972 - loss: 0.0085 - val_accuracy: 0.9878 - val_loss: 0.0480
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9973 - loss: 0.0080 - val_accuracy: 0.9917 - val_loss: 0.0377
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9977 - loss: 0.0068 - val_accuracy: 0.9913 - val_loss: 0.0397
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.9927 - loss: 0.0292
Test Accuracy: 0.9926999807357788
```




```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
```

```
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Acc (batch={batch})')
    plt.plot(histories[batch].history['val_accuracy'], linestyle='--', label=f'Val Acc (batch={batch})')

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 46s 27ms/step - accuracy: 0.9572 - loss: 0.1384 - val_accuracy: 0.9867 - val_loss: 0.0444
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 47s 28ms/step - accuracy: 0.9859 - loss: 0.0452 - val_accuracy: 0.9872 - val_loss: 0.0386
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 44s 26ms/step - accuracy: 0.9902 - loss: 0.0305 - val_accuracy: 0.9890 - val_loss: 0.0359
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 46s 28ms/step - accuracy: 0.9928 - loss: 0.0225 - val_accuracy: 0.9915 - val_loss: 0.0305
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 44s 26ms/step - accuracy: 0.9950 - loss: 0.0156 - val_accuracy: 0.9910 - val_loss: 0.0365
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 83s 26ms/step - accuracy: 0.9957 - loss: 0.0124 - val_accuracy: 0.9913 - val_loss: 0.0345
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 44s 26ms/step - accuracy: 0.9964 - loss: 0.0105 - val_accuracy: 0.9908 - val_loss: 0.0473
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 47s 28ms/step - accuracy: 0.9974 - loss: 0.0074 - val_accuracy: 0.9917 - val_loss: 0.0397
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 45s 26ms/step - accuracy: 0.9974 - loss: 0.0077 - val_accuracy: 0.9923 - val_loss: 0.0388
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 82s 26ms/step - accuracy: 0.9982 - loss: 0.0059 - val_accuracy: 0.9923 - val_loss: 0.0445

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 48ms/step - accuracy: 0.9487 - loss: 0.1683 - val_accuracy: 0.9837 - val_loss: 0.0573
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9837 - loss: 0.0530 - val_accuracy: 0.9850 - val_loss: 0.0490
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 49ms/step - accuracy: 0.9891 - loss: 0.0350 - val_accuracy: 0.9888 - val_loss: 0.0369
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9924 - loss: 0.0245 - val_accuracy: 0.9895 - val_loss: 0.0358
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - accuracy: 0.9936 - loss: 0.0199 - val_accuracy: 0.9888 - val_loss: 0.0405
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9949 - loss: 0.0156 - val_accuracy: 0.9915 - val_loss: 0.0361
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9959 - loss: 0.0126 - val_accuracy: 0.9917 - val_loss: 0.0370
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - accuracy: 0.9966 - loss: 0.0105 - val_accuracy: 0.9923 - val_loss: 0.0368
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 46ms/step - accuracy: 0.9968 - loss: 0.0089 - val_accuracy: 0.9910 - val_loss: 0.0397
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9978 - loss: 0.0064 - val_accuracy: 0.9888 - val_loss: 0.0532

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 39s 90ms/step - accuracy: 0.9336 - loss: 0.2369 - val_accuracy: 0.9802 - val_loss: 0.0749
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 38s 89ms/step - accuracy: 0.9800 - loss: 0.0626 - val_accuracy: 0.9832 - val_loss: 0.0574
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 41s 90ms/step - accuracy: 0.9867 - loss: 0.0433 - val_accuracy: 0.9880 - val_loss: 0.0424
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 38s 90ms/step - accuracy: 0.9902 - loss: 0.0314 - val_accuracy: 0.9883 - val_loss: 0.0426
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 38s 90ms/step - accuracy: 0.9923 - loss: 0.0251 - val_accuracy: 0.9908 - val_loss: 0.0342
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 38s 89ms/step - accuracy: 0.9942 - loss: 0.0190 - val_accuracy: 0.9880 - val_loss: 0.0388
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Flatten, Dense
```

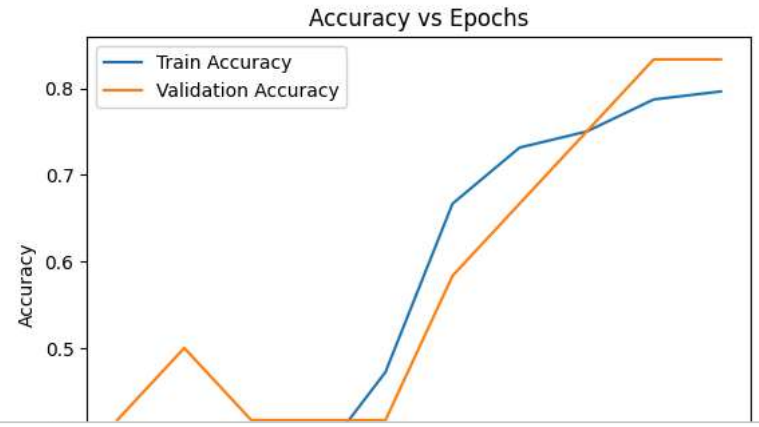
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Normalize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Convert to "image" shape (2x2x1)
X = X.reshape(-1, 2, 2, 1)

# One-hot encode labels
y = to_categorical(y, 3)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build CNN model
model = Sequential([
    Conv2D(32, (2,2), activation='relu', input_shape=(2,2,1)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Summary
model.summary()

# Train (STORE HISTORY)
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# =========================
# 📊 PLOT GRAPHS
# =========================

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Accuracy vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Loss vs Epochs')
plt.xlabel('Epochs')
```

```
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 1, 1, 32)	160
flatten (Flatten)	(None, 32)	0
dense (Dense)	(None, 128)	4,224
dense_1 (Dense)	(None, 3)	387

```
 Total params: 4,771 (18.64 KB)
 Trainable params: 4,771 (18.64 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.3148 - loss: 1.1412 - val_accuracy: 0.4167 - val_loss: 1.1147
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step - accuracy: 0.3148 - loss: 1.1190 - val_accuracy: 0.5000 - val_loss: 1.1016
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 0.3148 - loss: 1.0972 - val_accuracy: 0.4167 - val_loss: 1.0886
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.3704 - loss: 1.0760 - val_accuracy: 0.4167 - val_loss: 1.0762
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - accuracy: 0.4722 - loss: 1.0552 - val_accuracy: 0.4167 - val_loss: 1.0639
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - accuracy: 0.6667 - loss: 1.0348 - val_accuracy: 0.5833 - val_loss: 1.0517
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.7315 - loss: 1.0149 - val_accuracy: 0.6667 - val_loss: 1.0394
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - accuracy: 0.7500 - loss: 0.9953 - val_accuracy: 0.7500 - val_loss: 1.0272
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.7870 - loss: 0.9760 - val_accuracy: 0.8333 - val_loss: 1.0150
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step - accuracy: 0.7963 - loss: 0.9571 - val_accuracy: 0.8333 - val_loss: 1.0028
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - accuracy: 0.9000 - loss: 0.9297
Test accuracy: 0.8999999761581421
```




```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =========================
# LOAD CSV DATASET
# =========================
df = pd.read_csv('/content/Irisdl.csv')

# Check structure
print(df.head())

# Assuming last column is label
# Corrected: Exclude the 'Id' column (first column) and the 'Species' column (last column) to get 4 features.
```

```
X = df.iloc[:, 1:-1].values
y = df.iloc[:, -1].values

# ========================
# ENCODE LABELS (if needed)
# ========================
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)

# One-hot encode
y = to_categorical(y, 3)

# ========================
# NORMALIZE FEATURES
# ========================
scaler = StandardScaler()
X = scaler.fit_transform(X)

# ========================
# RESHAPE FOR CNN (2x2x1)
# ========================
X = X.reshape(-1, 2, 2, 1)

# ========================
# TRAIN-TEST SPLIT
# ========================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ========================
# BUILD MODEL
# ========================
model = Sequential([
    Conv2D(32, (2,2), activation='relu', input_shape=(2,2,1)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Summary
model.summary()

# ========================
# TRAIN
# ========================
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=16,
    validation_split=0.1
)

# ========================
# EVALUATE
# ========================
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# ========================
# PLOT GRAPHS
# ========================

# Accuracy
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Accuracy vs Epochs')
plt.xlabel('Epochs')
```

```
    plt.ylabel('Accuracy')
    plt.legend(['Train Accuracy', 'Validation Accuracy'])
    plt.show()
```